Trabajo Practico - Taller de Programación

Carlos Castillo - 108535
Germán Douce - 106001
Lucas Araujo - 109867

2025 1er Cuatrimestre

Índice

1. Introducción **3**

2. Estructura del repositorio **3**

3. Servidor **4**

4. Nodo **5**
4.1. Protocolo RESP 6

5. Almacenamiento **6**
5.1. Persistencia en Disco 7

6. Pub/Sub **7**
6.1. Conexiones persistentes y modelo de ejecución 7
6.2. Manejo de clientes y concurrencia 7
6.3. Broker centralizado para evitar condiciones de carrera 7
6.4. División del stream en lectura y escritura 8
6.5. Integración con el clúster 8
6.6. Aplicaciones del sistema 8

7. Cluster **8**
7.1. Gossip 10
7.2. Mensajes implementados 11
7.3. Replicación 12
7.4. Failover y replica promotion 13
7.4.1. Escenario #1: ningún nodo alcanza el quórum 13
7.4.2. Escenario #2: un nodo gana la elección antes que otro 13
7.4.3. Escenario #3: empate o conflicto en la elección 14
7.4.4. Detección de fallas 14

8. Aplicaciones cliente **15**
8.1. Protocolo de mensajes 15
8.2. Microservicio de control y persistencia 16
8.3. Microservicio de generación de contenido 17
8.3.1. Elección del proveedor de IA 17
8.3.2. Arquitectura interna 17
8.3.3. Solución multithreading adoptada 18
8.3.4. Uso de la API de OpenAI 19
8.4. Editor de texto colaborativo 19
8.4.1. Pantalla de edición 20
8.4.2. Generación de contenido 21

9. Rendimiento y costos de funcionamiento **21**

10. Aprendizajes obtenidos **23**

1. Introducción

El proyecto consiste en la implementación de un servidor Redis, junto con dos microservicios cliente y un editor de documentos colaborativo. El servidor Redis implementado soporta funcionalidades como mensajes pub/sub, modo de clúster para distribuir la base de datos, persistencia en disco y otras características adicionales, como soporte para cifrado de conexiones y configuración del servidor.

Los dos microservicios, uno de control y persistencia y otro de generación de documentos, complementan el funcionamiento del editor de texto colaborativo, que cuenta con una interfaz gráfica básica que permite a múltiples usuarios visualizar en tiempo real los cambios en un mismo documento. El microservicio de control proporciona a los clientes el contenido de los documentos, actualiza el listado de usuarios conectados y persiste en la base de datos el contenido editado. El microservicio de generación de contenido contacta a un proveedor de LLM para resolver las solicitudes de generación de texto de los usuarios.

La implementación tanto del servidor como de las aplicaciones clientes se realizó en el lenguaje de programación Rust.

2. Estructura del repositorio

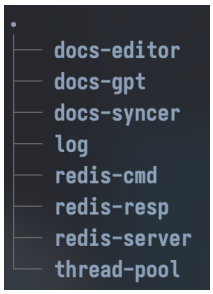

Figura 1: Estructura del repositorio

El proyecto se encuentra organizado utilizando un workspace de Cargo, en donde se incluyen tanto los crates binarios del servidor y los clientes, como las librerías comúnes utilidas por múltiples crates del workspace. La descripción de los crates es la siguiente:

- `docs-editor`: Editor de documentos colaborativo.
- `docs-gpt`: Microservicio de generación de contenido.
- `docs-syncer`: Microservicio de control y persistencia.
- `log`: Librería para unificar el formato de los logs a través de todos los crates del workspace.
- `redis-cmd`: Librería para la serialización y deserialización de comandos compatibles con el servidor.
- `redis-resp`: Librería para la serialización y deserialización del protocolo RESP.
- `redis-server`: Implementación del servidor Redis.
- `thread-pool`: Implementación de una thread pool.

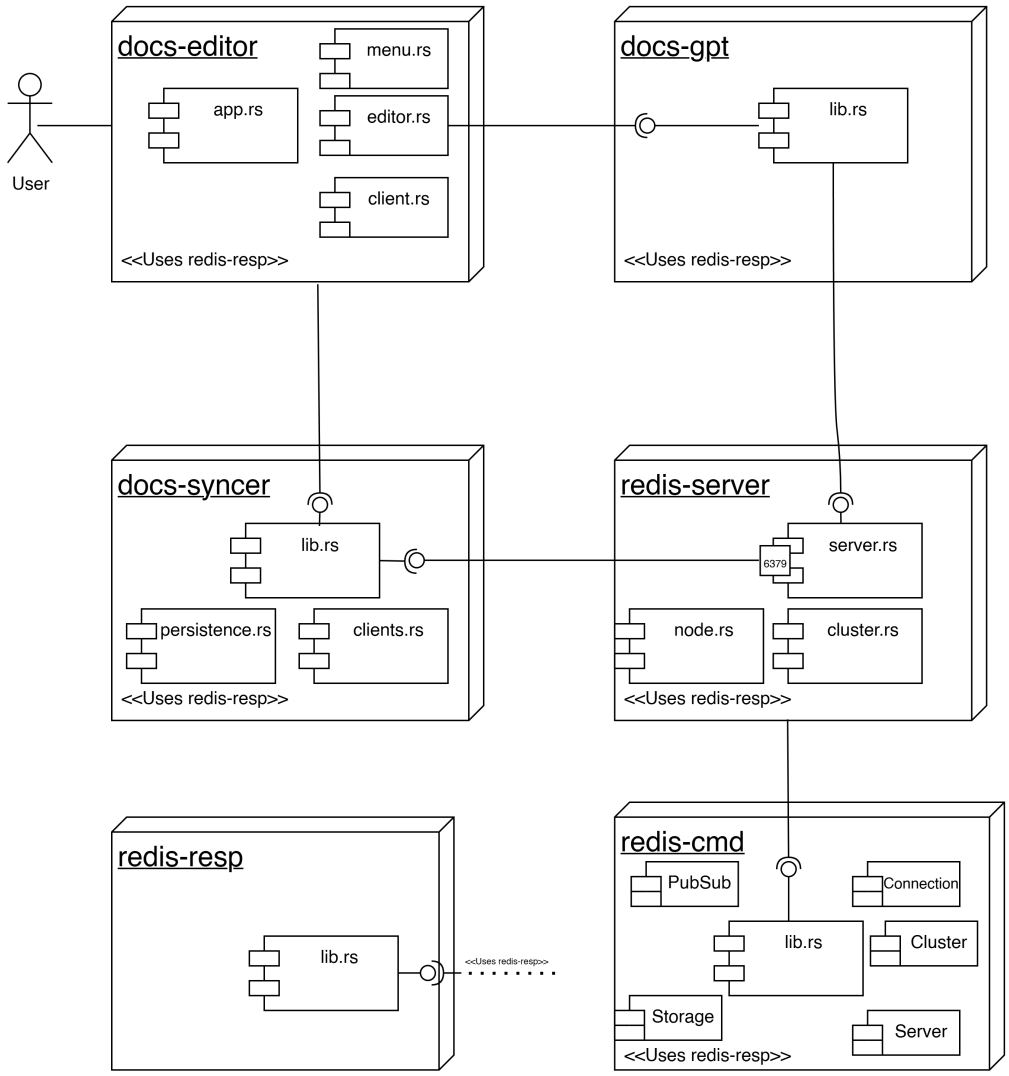

Figura 2: Diagrama de despliegue que ilustra la arquitectura general del sistema

El sistema está compuesto por múltiples módulos independientes, como `docs-editor`, `docs-gpt` y `docs-syncer`, que se comunican entre sí a través de Redis con el protocolo `redis-resp`.

3. Servidor

La responsabilidad del servidor de Redis es manejar las consultas que envían los clientes mediante conexiones TCP. Para cada cliente que se conecta, el servidor toma un hilo disponible desde la thread pool. Dicho hilo retornará a la thread pool una vez haya sido enviada la respuesta al cliente solicitante. Esto permite el manejo de múltiples solicitudes en simultáneo. Tras consultar las diferentes alternativas posibles con nuestro equipo, optamos por implementar un diseño que manejará las conexiones con los clientes de manera efímera. Es decir, se envía solamente un comando por conexión, salvo en el caso del comando `Subscribe`, en donde la conexión con el cliente permanece abierta indefinidamente hasta que el cliente se desconecta de forma intencional o se produce un error en la comunicación.
Internamente, el servidor utiliza una dirección IP y un puerto de escucha configurables, un *thread pool* que facilita la ejecución concurrente de las tareas, y un canal de envío

de mensajes de log al sistema de registro de eventos. Cada conexión aceptada por el `TcpListener` se delega en el método `handle_client` del nodo, ejecutado en un hilo del *thread pool*. Si durante el manejo de la conexión se produce un error, este se captura y se envía al canal de logs para su registro. Este diseño permite mantener un flujo de procesamiento estable, registrar los errores en tiempo real y sostener conexiones prolongadas con los clientes en operaciones que así lo requieren.

4. Nodo

Una vez la solicitud entra al servidor, es la estructura Nodo la que delega el manejo del comando enviado a las diferentes componentes funcionales que conforman el nodo de Redis. Nuestra implementación soporta múltiples tipos de comandos, como comandos de tipo pub/sub, comandos relacionados al almacenamiento, o comandos relacionados a la funcionalidad del cluster. Para cada uno de estas categorías, el nodo cuenta con un subsistema que va a ser el que se encarga de cada comando de acuerdo a su categoría.

Todos estos subsistemas implementan un diseño en el que son ejecutados cada uno en su propio hilo, por lo que la comunicación entre desde y hacia los mismos se hace mediante el pasaje de mensajes utilizando canales de concurrencia. en la mayoría de los casos, esta decisión de diseño facilita considerablemente el acceso a los datos que deben estar centralizados, como los datos en memoria par el almacenamiento, el registro de subscripciones y canales, o el estado del nodo y la visión del cluster para la funcionalidad distribuída.

```rust
pub struct Node {
    broker_tx: Sender<PubSubEnvelope>,
    storage_tx: Sender<storage::StorageAction>,
    replication_tx: Sender<ReplicationAction>,
    cluster_tx: Option<Sender<ClusterAction>>,
    pub_sub_tx: Sender<PublishPayload>,
    log_tx: Sender<Log>,
}
```

Figura 3: Estructura del nodo

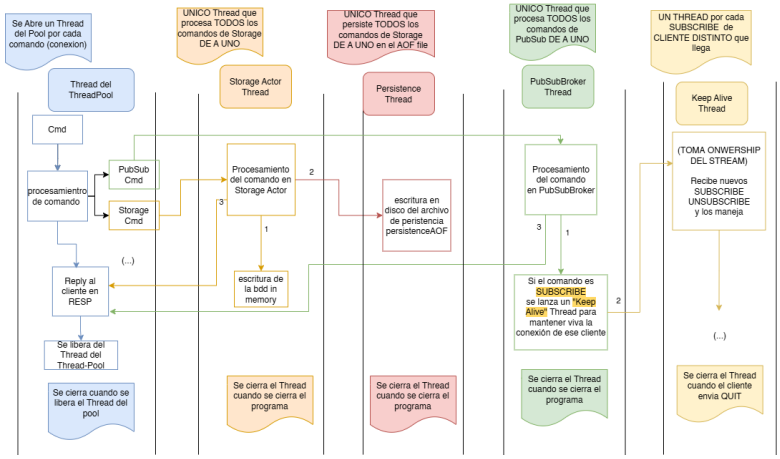

Figura 4: Diagrama de secuencia de la ejecución de comandos

Finalmente, los subsistemas notifican al nodo que ya se ha procesado la solicitud y se ha recibido una respuesta, por lo que el nodo, quien tiene control sobre la conexión TCP con el cliente, envía la respuesta al mismo y termina el procesamiento de la solicitud.

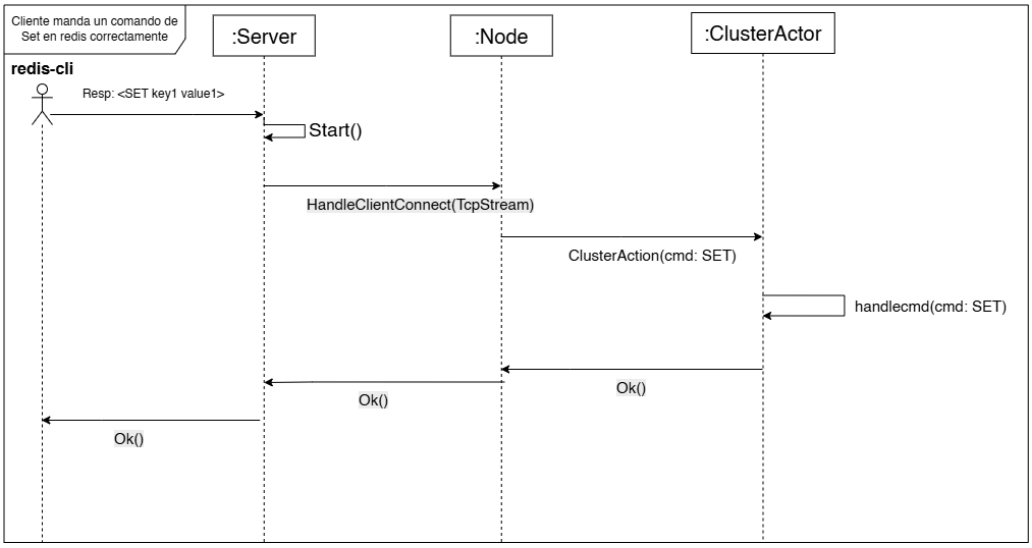

Figura 5: Diagrama de secuencia del procesamiento del comando `SET`.

4.1. Protocolo RESP

Esta implementación hace uso de un subconjunto de tipos de datos soportados por el protocolo RESP 3: `array`, `boolean`, `bulk string`, `double`, `integer`, `map`, `null`, `set`, `simple error` y `simple string`.

Se optó por esta versión del protocolo por sobre la versión 2 ya que, a pesar de ser más compleja de implementar, provee tipos de datos que consideramos más expresivos, como es el caso de los map y los sets, contrario a RESP 2, que representa estos tipos de datos como array. Consideramos que esta mayor expresividad iba a ser de utilidad para los clientes de nuestro servidor.

5. Almacenamiento

Para manejar la funcionalidad de almacenamiento de datos en la base de datos en memoria y la persistencia en disco se utilizaron dos subsistemas llamados `StorageActor` y `PersistenceActor` respectivamente.

La centralización de los datos en un único actor que corre en una thread aparte y que recibe solicitudes de acceso para lectura y modificación de datos, permitió que internamente el código fuese mucho más simple de escribir y mantener, así como también permitió evitar el uso de mutexes para el manejo del estado de la base de datos. En el diseño elegido, las múltiples threads que manejan las conexiones de los clientes, mandan sus solicitudes al `StorageActor` a través del canal de comunicación único hacia esta actor, quien luego responde a cada thread por sus respectivos canales con la respuesta del comando ejecuta ya serializada.

Para la representación interna de los tipos de datos soportados por los comandos implementados, se optó por utilizar la siguiente estructura:

```rust
pub enum RedisDataType {
    String(BulkString),
    Hash(HashMap<BulkString, BulkString>),
    List(LinkedList<BulkString>),
    Set(HashSet<BulkString>),
}
```

Figura 6: Estructura de datos en memoria

La decisión de utilizar listas enlazadas para la implementación del tipo de dato de listas surge de cómo la implementación oficial de Redis las implementa internamente. Respecto a los otros tipos de datos, se optó por utilizar la estructura de datos más simple para reproducir su funcionalidad.

5.1. Persistencia en Disco

Se optó por utilizar el protocolo de serialización append-only file (AOF, por sus siglas en inglés), debido a la facilidad de implementación una vez se había terminado de implementar el subconjunto de tipos soportados por el protocolo RESP 3.

La persistencia en disco se hace únicamente para los comandos que modifican el estado de la base de datos, no para aquellos que solo realizan consultas de lectura. Además, se soporta la reconstrucción del último estado persistido de la base de datos de manera automática al iniciar el nodo si el archivo de persistencia configurado está disponible, si no, entonces se inicializa uno nuevo.

6. Pub/Sub

En nuestro proyecto en Rust implementamos un sistema de publicación y suscripción (Pub/Sub) robusto y escalable, con soporte tanto a nivel local como distribuido dentro del clúster. Este sistema fue clave para construir un protocolo de mensajería personalizado que permite la comunicación fluida entre editores y microservicios, así como la sincronización en tiempo real del documento colaborativo sobre el que trabajan múltiples usuarios.

6.1. Conexiones persistentes y modelo de ejecución

Una de las características fundamentales de nuestra implementación es que las conexiones de los clientes no son efímeras cuando un cliente se suscribe a un determinado canal. En lugar de cerrarse tras recibir dicho mensaje, se mantienen abiertas mediante un thread que llamamos “keep alive”, lo que permite que los clientes puedan seguir recibiendo mensajes por el canal al que se suscribieron y enviando nuevos comandos (como nuevas suscripciones o desuscripciones) sin necesidad de reconectar.

6.2. Manejo de clientes y concurrencia

Para cada cliente que se suscribe a uno o varios canales, se lanza un thread independiente que para atender a este cliente. Este thread se encarga de enviarle los mensajes que se publiquen en los canales a los que está suscripto y de recibir nuevos comandos que el cliente envíe. Si el cliente se desconecta o se desuscribe de todos los canales, el thread correspondiente se cierra, liberando los recursos asociados.

6.3. Broker centralizado para evitar condiciones de carrera

El corazón del sistema es el `PubSubBroker`, implementado como un actor. Este componente es el único que tiene acceso al registro global de suscripciones, lo que nos permitió

evitar condiciones de carrera (race conditions) y mantener un control centralizado y seguro sobre qué clientes están suscritos a qué canales.

6.4. División del stream en lectura y escritura

Al momento de procesar la conexión de cada cliente, dividimos su stream TCP en stream de lectura y otro de escritura. Esto nos permite escuchar comandos del cliente (como `Subscribe`, `Unsubscribe`, etc.) mientras simultáneamente le enviamos mensajes que provienen de los canales a los que está suscrito, logrando una comunicación bidireccional eficiente.

6.5. Integración con el clúster

Nuestra implementación del Pub/Sub funciona también a nivel de clúster, lo cual fue clave para soportar alta disponibilidad y distribución. Cuando un mensaje es publicado en un nodo del clúster, este se reenvía a los demás nodos mediante nuestro protocolo interno de clúster usando un mensaje especial de tipo Publish. Al recibirlo, cada nodo escribe el mensaje en el canal correspondiente dentro de su propio `PubSubBroker`, permitiendo que los clientes conectados a distintos nodos del clúster puedan recibir los mensajes en tiempo real.

6.6. Aplicaciones del sistema

Todo este sistema de Pub/Sub permite soportar dos funcionalidades principales en nuestra arquitectura:

- El protocolo de mensajería interno, que permite la comunicación entre el editor del usuario y los microservicios que procesan eventos relacionados al documento.
- La sincronización en tiempo real del documento compartido, ya que todos los usuarios que están editando el mismo documento están conectados a un canal común, lo que asegura que los cambios se reflejen instantáneamente entre todos los participantes.

7. Cluster

Una de las principales dificultades al definir cómo abordar la implementación de la funcionalidad de cluster, fue definir la forma en cómo tenía que gestionar cada nodo su estado. Cada nodo en cluster tiene tanto información propia, como información de los otros nodos del cluster a los que conoce, información que denominamos `cluster_view`.

Muchas de la información del nodo tiene que ir mutando como respueta a diferentes eventos que suceden internamente en cada nodo y a nivel cluster, por ejemplo, se debe actualizar la visión del cluster al conocer un nuevo nodo tras recibir un mensaje, pero en este mismo suceso puede también mutar el epoch del nodo receptor, o, en un mensaje de votación, por ejemplo, se puede tener que actualizar el contador de votos interno también, etc.

Para evitar el uso de mutexes y facilitar la edición de la información del nodo, se decidió, nuevamente, optar por una arquitectura basada en el modelo de actores, en el cual todas las acciones que deban modificar parámetros del nodo y su visión del cluster, ahora encapsulados en la entidad `ClusterActor`, se encolan en un único thread que responde a estas acciones que se le solicita, procesándolas y ejecutándolas una a la vez garantizando la seguridad de un entorno concurrente.

Cada evento relevante para el clúster como comandos de clientes, mensajes entre nodos, detección de fallos, publicaciones o redirecciones se encapsula en un `ClusterAction` y se envía a un canal que es procesado por un único hilo de ejecución.

```rust
pub struct ClusterActor {
    myself: ClusterNode,
    current_epoch: u64,
    last_vote_epoch: u64,
    cluster_view: HashMap<NodeId, ClusterNode>,
    cluster_streams: HashMap<NodeId, TcpStream>,
    timeout_millis: u64,
    failure_reports: HashMap<NodeId, HashMap<NodeId, FailureReport
        >>,
    replication_link: Option<TcpStream>,
    storage_tx: Sender<StorageAction>,
    pub_sub_tx: Sender<PublishPayload>,
    votes_received: u64,
    failover_epoch: Option<u64>,
    failover_start: Option<SystemTime>,
    failover_in_progress: bool,
    failover_timeout_millis: u64,
}
```

Figura 7: Estructura del Cluster Actor

```rust
pub enum ClusterAction {
    ClientCommand {
        cmd: ClusterCommand,
        reply_tx: Sender<Vec<u8>>,
    },
    ClusterMessage(Message),
    TickPing,
    CheckFailures,
    ConfirmFailure {
        id: NodeId,
    },
    BroadcastPublish {
        channel: BulkString,
        message: BulkString,
    },
    RedirectToHoldingNode {
        key: BulkString,
        redir_tx: Sender<Option<Vec<u8>>>,
    },
}
```

Figura 8: Listado de `ClusterAction`

Cada `ClusterAction` define una operación de clúster, que pueden ser una de las siguientes:

- `ClientCommand`: Para comandos de cliente que necesitan ser manejados por el clúster, por ejemplo, redirección de claves a los nodos correctos.
- `ClusterMessage`: Para intercambiar mensajes internos del clúster, como mensajes de *gossip*.
- `TickPing` y `CheckFailures` Utilizados para la detección de fallas y el mantenimiento de la *liveness* del clúster. Se envían de manera periódica.
- `ConfirmFailure`: Para notificar y confirmar la falla de otros nodos dentro del clúster.

- `BroadcastPublish`: Permite difundir mensajes de tipo `PUBLISH` a través de todo el clúster, asegurando que todos los suscriptores reciban el mensaje sin importar a qué nodo estén conectados.
- `RedirectToHoldingNode`: Esencial para el funcionamiento del clúster, redirige solicitudes de claves a los nodos maestros que poseen esos datos, incluyendo la capacidad de responder al cliente con la dirección correcta.

7.1. Gossip

El cluster consiste en una red de servidores distribuídos, o nodos, cada uno ejecutándose como un proceso diferente, con una dirección de red distinta. Al no haber un orquestrador central del cluster, los mismos nodos son quienes deben organizarse entre sí para trabajar en conjunto. Esta fue la primera vez que todos los miembros del grupo trabajaron con este paradigma, por lo que tuvimos que aprender a cambiar el enfoque de trabajo para adaptarnos a esta nueva forma de trabajar, siendo el aprender a cómo lidiar con la asincronía de las operaciones la parte más difícil del proceso. La clave de cómo terminamos entendiendo el funcionamiente del sistema distribuído radica en el concepto del pasado de mensajes que utiliza Redis Cluster, y su idea central, los mensajes de gossip.

Los mensajes de gossip son esenciales para la comunicación continua entre los nodos del clúster. A través de este mecanismo, cada nodo intercambia información periódica sobre el estado de los demás, incluyendo qué nodos están activos, qué roles cumplen (maestro o réplica), y la configuración general del clúster.

Este intercambio constante permite que cada nodo mantenga una vista parcialmente actualizada del clúster, que converge con el tiempo hacia un estado globalmente coherente. Aunque los cambios no se propagan de forma instantánea, eventualmente todos los nodos terminan compartiendo la misma información sobre la topología y el estado del sistema. Por eso, el modelo que se obtiene es de consistencia eventual: dado suficiente tiempo y sin nuevos cambios, todos los nodos alcanzarán el mismo conocimiento sobre la estructura y el estado del clúster.

Este enfoque es fundamental para la tolerancia a fallos del sistema. Permite, por ejemplo, detectar nodos caídos, elegir nuevos maestros o sin necesidad de una autoridad central ni coordinación estricta.

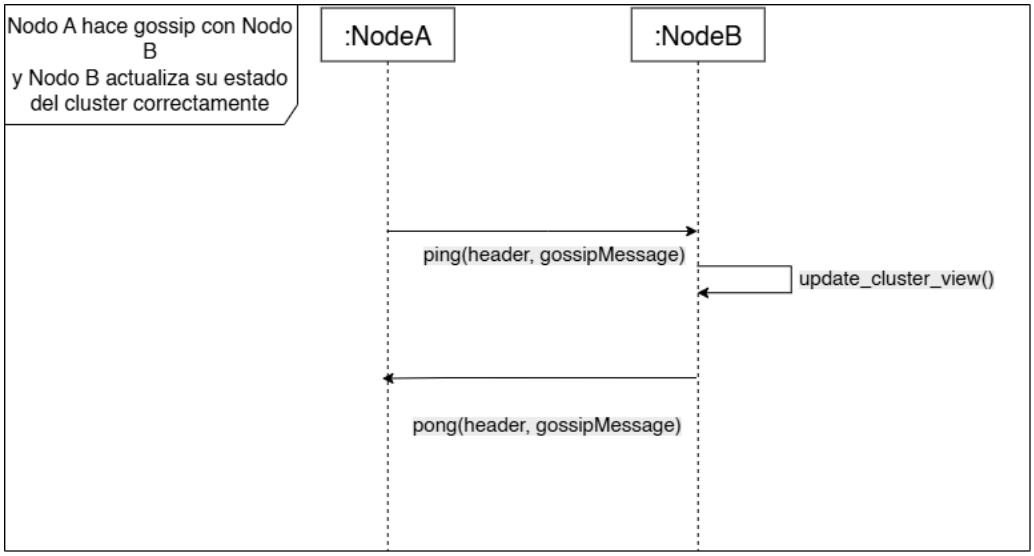

Figura 9: Nodo A actualiza el estado del cluster a través de `PING` de Nodo B

7.2. Mensajes implementados

Los tipos de mensajes implementados son los siguientes:

- `MEET`: Utilizado para que un nodo nuevo que no es parte del cluster anuncie su existencia a un solo nodo del cluster y por medio de gossip todo el cluster se entere de la existencia de este nuevo nodo. De esta manera ese nuevo nodo se unio al cluster.
- `PING` y `PONG`: Enviado periódicamente cada 1 segundo entre nodos para verificar su disponibilidad y para intercambiar información de estado. Los nodos responden con un `PONG` para indicar que están activos.
- `FAIL`: Un mensaje enviado por un nodo cuando ha detectado que otro nodo ha fallado (se ha vuelto inaccesible o no responde). Este reporte contribuye al consenso sobre la falla de un nodo.
- `AUTH_REQUEST`: Utilizado en el proceso de *failover*. Una réplica que ha detectado la falla de su maestro puede enviar este mensaje para solicitar votos a otros nodos y convertirse en el nuevo maestro. El `ClusterActor` gestiona el inicio de *failovers*, incrementando su `current_epoch` y enviando solicitudes de *failover* si un maestro no es restaurado y el nodo actual no es un maestro.
- `AUTH_ACK`: La respuesta de un nodo a un AuthRequest, indicando que otorga su voto a la réplica solicitante en el proceso de elección de un nuevo maestro.
- `PUBLISH`: Permite la difusión de mensajes a través del sistema de publicación/suscripción del clúster. Cuando un mensaje es publicado, este se propaga a todos los nodos, asegurando que cualquier cliente suscrito lo reciba, independientemente del nodo al que esté conectado.
- `UPDATE`: Un mensaje para comunicar actualizaciones de configuración del clúster o cambios en la asignación de slots. Se utiliza para mantener rápidamente la consistencia del estado del clúster en todos los nodos.

Todos los tipos de mensaje comparten un encabezado común, que proporciona información propia adicional del nodo que envía el mensaje. Así, por ejemplo, en un mensaje de `PING` donde un nodo comparte información de su visión del cluster con otros nodos, el nodo también va a compartir información sobre sí mismo a través de este header. Esta información va a incluir información adicional a la compartida sobre los nodos seleccionados de la cluster view, como por ejemplo los slots que maneja dicho nodo enviador o el id de su master, si es que cuenta con uno.

La decisión de optar por un diseño así fue inspirada por la implementación oficial de los mensajes de cluster de Redis.

```rust
pub struct MessageHeader {
    pub id: NodeId,
    pub ip: Ipv4Addr,
    pub port: u16,
    pub cluster_port: u16,
    pub flags: Flags,
    pub config_epoch: u64,
    pub slots: u16,
    pub master_id: Option<NodeId>,
}
```

Figura 10: Encabezado común de todos los mensajes de cluster.

Además, esto permite a cada nodo distinguir entre la veracidad de la información que otros nodos le estan compartiendo. Cada nodo sabe que la infomración compartida en el encabezado de un mensaje enviado de un nodo dado, es siempre la versión más actualizada de la información de ese nodo, ya que está siendo enviada por el nodo mismo.

```
pub struct GossipNode {
    pub id: NodeId,
    pub ip: Ipv4Addr,
    pub port: u16,
    pub cluster_port: u16,
    pub flags: Flags,
}
```

Figura 11: Información compartida sobre otros nodos a través de gossip.

7.3. Replicación

Cuando a un nodo se le asigna un master, ya se porque se hizo de manera manual mediante la ejecución del comando `CLUSTER REPLICATE` o porque terminó un proceso de elección, el nodo que queda como réplica de un nuevo master.
Para que la replicación funcione de manera correcta, se debe realizar una sincronizacion de cada réplica con su master, para que las réplicas que no tienen datos cargados con los datos que guardo el master hasta el momento, reciban un envío inicial con el estado actual de la base de datos del master. En nuestro caso, como utilizamos persistencia AOF, el estado de la base de datos puede ser reconstruído a partir del historial de comandos ejecutados.

Por la separación de subsistemas que implementamos en el nodo, es el `ClusterActor` quien tiene control sobre el estado de los roles de un nodo, y es otro actor, el `StorageActor`, quien tiene control sobre el estado y persistencia de la base de datos. Para solucionar esto se tuvo que implementar un sistema de comunicación entre los diferentes componentes del nodo. Sistema que, nuevamente, utiliza pasaje de mensajes como vía de comunicación.

```
pub enum ReplicationAction {
    BroadcastCommand { bytes: Vec<u8> },
    SyncReplica { stream: TcpStream },
}
```

Figura 12: Acciones enviadas entre `ClusterActor` y `StorageActor` para replicación de masters.

Cuando se define el rol de un nodo como réplica de otro, el `StorageActor` procesa el comando `SYNC` que inicia la primera fase de sincronización de datos: la sincronización inicial o full sync.

En la sincronización inicial, el nodo maestro envía a la réplica una copia completa de todos los comandos almacenados hasta ese momento en su sistema de almacenamiento, es decir, envía su AOF actual. De este modo, la réplica puede ponerse al día y quedar en un estado coherente con el nodo maestro. Es la réplica quien le notifica al master que desea iniciar la sincronización, utilizando la acción `SyncReplica`. A partir de este momento, se abre una conexión exclusiva para la replicación entre master y réplica.

Una vez que la réplica ya está sincronizada, el sistema entra en una fase de replicación continua. Cada vez que el nodo maestro ejecuta un nuevo comando, este se transmite inmediatamente a sus réplicas. De esta manera, las réplicas permanecen actualizadas en tiempo real con las operaciones que se van ejecutando. En este caso, es el `StorageActor`

quien inicia el proceso de publicación de comandos a sus réplicamos, mediante la acción `BroadcastCommand`.

7.4. Failover y replica promotion

La promoción de réplica ocurre cuando un nodo que actúa como réplica detecta que su maestro ha fallado. Este nodo iniciará un proceso de *failover*, incrementando su `current_epoch` y votándose a sí mismo. Luego, solicitará votos a otros nodos maestros activos en el clúster.
Cuando una réplica recibe suficientes votos de otros nodos (alcanzando un quórum basado en el número de maestros activos), y no hay otro maestro ya ascendido para sus *slots*, se declarará a sí misma como el nuevo maestro. En este punto, actualiza sus propias flags para reflejar su nuevo rol con `FLAG_MASTER`, elimina cualquier `master_id` anterior y establece su `config_epoch` al `current_epoch` de la elección.
Si, durante este proceso, la réplica detecta que ya existe un maestro para sus *slots* (posiblemente uno que se adelantó en la promoción y cuya información se propagó por *gossip*), se configurará como réplica de ese maestro existente y el proceso de *failover* terminará para ese nodo.

7.4.1. Escenario #1: ningún nodo alcanza el quórum

Este escenario suele manifestarse cuando los nodos detectan que su maestro fallo pero aún no se ha propagado de lo largo del Clúster la información sobre el nodo que esta en ‘fail‘. Esto genera que los maestros no voten a ninguna réplica porque para ellos el nodo no esta caído.

Manejo en la Implementación:

- Al iniciarse una elección (failover), se activa un indicador interno: `failover_in_progress = true`.
- El bucle de verificación de fallos (`CheckFailures`, dentro de `handle_actions`) monitorea tanto la salud de los nodos como el estado del proceso de failover.
- Si transcurre un tiempo definido por `failover_timeout_millis` sin que la réplica logre la promoción, se reinicia el proceso: se incrementa el `current_epoch` y se vuelven a solicitar votos, generando una nueva “epoch” de elección.
- Este proceso se repite hasta que un nodo es elegido o se abandona explícitamente la elección.

7.4.2. Escenario #2: un nodo gana la elección antes que otro

Escenario ideal donde, si varias réplicas inician simultáneamente la elección, una logra alcanzar el quórum y se convierte en maestro antes que las demás.

Manejo en la Implementación:

- Antes de promocionarse, cada réplica consulta su `cluster_view` para verificar si ya existe un nodo maestro para los mismos slots.
- Si se detecta que otro nodo ya asumió el rol de maestro, la réplica desiste de su promoción.
- Esta verificación garantiza convergencia sin conflictos, permitiendo una elección ordenada.

7.4.3. Escenario #3: empate o conflicto en la elección

Situación crítica donde dos o más réplicas creen haber sido electas y se auto-promueven como maestras para los mismos rangos de slots.

Manejo en la Implementación:

- `handle_config_epoch_collision:` Cuando dos nodos anuncian el mismo `config_epoch`, se aplica una regla de desempate lexicográfica basada en el `NodeId`. El nodo con `NodeId` menor toma precedencia, incrementa su `config_epoch`, y prevalece como maestro. El otro no modifica nada.
- `check_for_same_slots:` Durante la propagación de estado vía Gossip, si dos nodos descubren que comparten slots idénticos:
  - Si el nodo local tiene un `config_epoch` menor (o mismo `epoch` pero `NodeId` mayor), se auto-despromueve (estableciendo `FLAG_SLAVE` y actualizando su `master_id`) con la del nodo que tiene sus mismos slots.
- Esto asegura que único nodo maneje cada rango de slots, previniendo la condición de *split-brain*.
- Los eventos relevantes quedan registrados mediante `log::warn!`, proporcionando trazabilidad clara sobre cómo se resolvió cada conflicto.

7.4.4. Detección de fallas

La detección de fallas en el clúster se basa en dos niveles: `PFAIL` ("probable falla") y `FAIL` ("falla confirmada").

- `PFAIL`: Un nodo marca a otro como `PFAIL` cuando no ha recibido señales de vida (por ejemplo, mensajes `PING` o `PONG`) dentro de un intervalo de tiempo determinado. Este estado es local y no implica una decisión global.
- `FAIL`: Para declarar un nodo como `FAIL`, múltiples nodos deben haber marcado al mismo nodo como `PFAIL` y comunicar esta sospecha a través de mensajes `FAIL`. Una vez que se alcanza consenso, el nodo es considerado oficialmente fallido y se inicia el proceso de failover si corresponde.

Este mecanismo de dos etapas evita falsas alarmas debidas a latencias momentáneas y asegura una detección distribuida y confiable de fallas.

8. Aplicaciones cliente

Los 3 clientes implementados para nuestro servidor de Redis son: el editor de documentos colaborativo, y los microservicios de control y persistencia, y de generación de contenido.

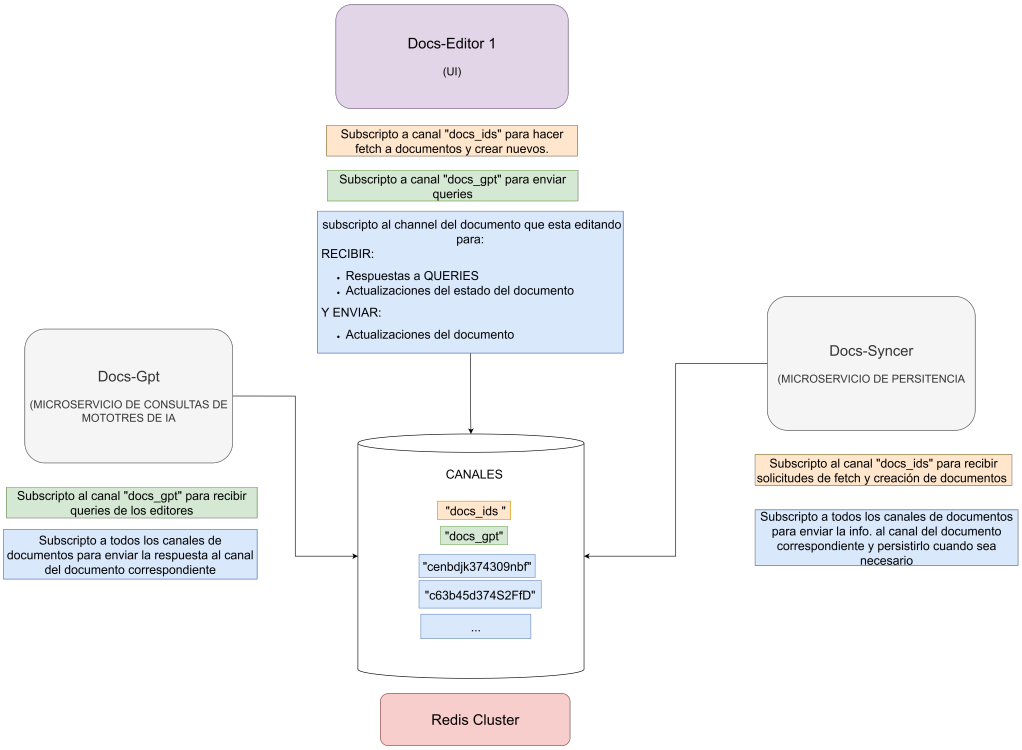

Figura 13: Funcionamiento en conjunto del editor, microservicios y Redis.

8.1. Protocolo de mensajes

Dado que ninguno de los tres clientes se comunica con los demás de manera directa, sino que lo hacen exclusivamente a través de canales de pub/sub, se optó por implementar un protocolo de mensajes para definir el formato de los mismo a través de los canales utilizados para la comunicación entre estos clientes. Este formato de mensajes permite a los microservicios recibir diferentes tipos de acciones de los editores, identificar qué editor está solicitando qué acción, y responder adecuadamente al editor correspondiente o a todos los editores en conjunto, en dependencia del mensaje. Utilizando el formato de mensajes de Redis ya comentado:
`PUBLISH <chan> <payload>`: Se publica un mensaje en un *canal*, donde el *payload* es el contenido del mensaje utilizado en el protocolo para que tanto los microservicios como el editor puedan enviarse solicitudes, datos o comunicar determinados eventos.

Los payloads (mensajes) que soporta el protocolo son los siguientes:

`PARTIALGEN_REQ` : Enviado por el cliente al microservicio `docs-gpt` para solicitar la generación parcial de texto.

`PARTIALGEN_ACK` : Respuesta de `docs-gpt` al cliente con el texto generado o un mensaje de error.

`FULLGEN_REQ` : Enviado por el cliente al microservicio `docs-gpt` para solicitar la generación completa de texto.

`FULLGEN_ACK` : Respuesta de `docs-gpt` al cliente con el texto generado o un mensaje de error.

`CLIENTS` : Mensaje del `docs-syncer` para informar a los editores que otros clientes hay conectados al documento.

`FETCH_REQ` : Solicitud del cliente editor a `docs-syncer` para obtener el estado actual guardado del documento.

`FETCH_ACK` : Respuesta de `docs-syncer` al cliente con la ultima versión guardada deldocumento.

`PATCH` : Enviado por el cliente editor para informar cambios locales en el documento. El microservicio lo lee y lo utiliza para persistir los cambios. Otros clientes editores lo leen y utilizan para actualizar la version del documento que tienen en memoria.

`LEAVE` : Enviado por el cliente editor para informar que se ha desconectado de un documento.

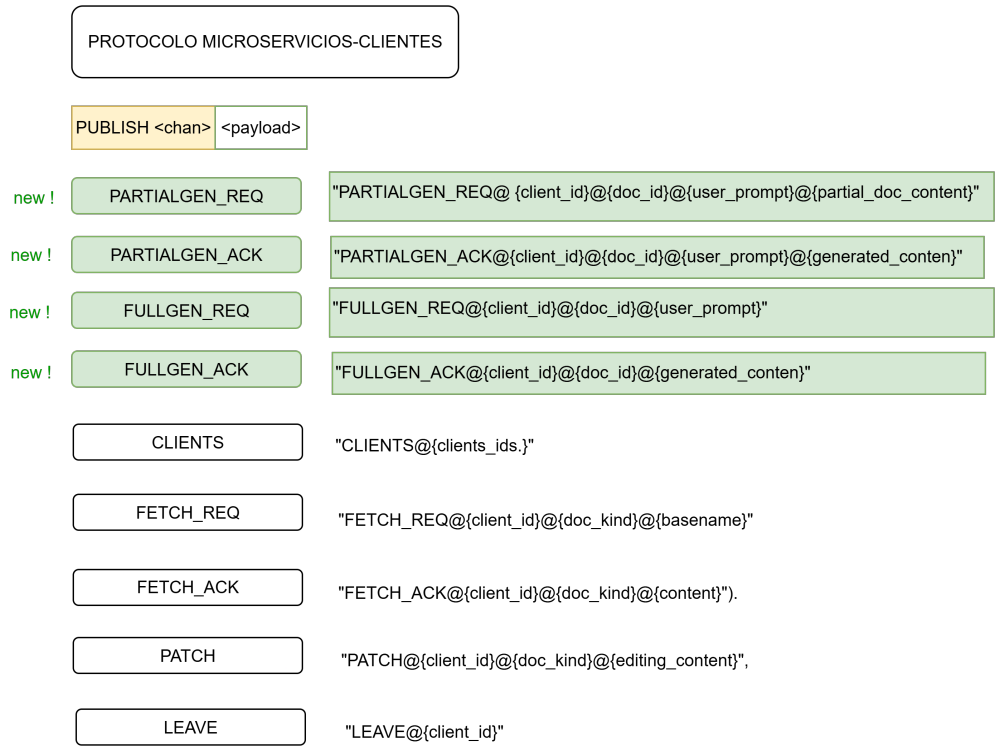

Figura 14: Mensajes soportados por el protocolo microservicios-clientes. Los marcados en verde se agregaron para la entrega final.

8.2. Microservicio de control y persistencia

Este microservicio se encarga de centralizar la gestión de documentos para la edición colaborativa. Actúa como intermediario entre los clientes y la base de datos, utilizando pub/sub para coordinar la creación y persistencia en tiempo real de los documentos y para informar que editores cliente se encuentran editando un documento en determinado momento. De esta manera, se desacopla la lógica de persistencia y de consulta de clientes activos de los editores.

Entre su funcionalidad clave encontramos:

- **Gestión del Ciclo de Vida del Documento:**
  - Se encarga de la *creación inicial de nuevos documentos*, persistiendo sus metadatos (ID y nombre base) en la base de datos.

- *Persiste los cambios* realizados en los documentos. Cuando un cliente editor modifica un documento y hace click en guardar, el `DocsSyncer` recibe estas guarda el documento con los cambios en nuestro redis server.

- **Pub/sub**
  - Actúa como un *intermediario de mensajería*. Está suscrito a un canal llamado *docs_ids*, por el que recibe mensajes para la creación de documentos. A su vez, está suscrito a todos los documentos que están editando los clientes para recibir actualizaciones de los mismos y persistir los cambios en la base de datos.

8.3. Microservicio de generación de contenido

El microservicio `docs-gpt` tiene como responsabilidad principal procesar solicitudes de generación o edición de texto mediante un modelo de lenguaje (LLM), utilizando la API de OpenAI. Actúa como un intermediario entre el usuario editando el contenido y este proveedor de IA, recibiendo los dos tipos de solicitudes soportados: parcial y completa, y devolviendo la respuesta el editor solicitante.

8.3.1. Elección del proveedor de IA

Algunos integrantes del grupo ya estaban familiarizados con el formato de las respuestas de este proveedor. Este proveedor ofrece una gran variedad de modelos que pudimos probar. Finalmente, optamos por un modelo con una mayor velocidad de respuesta pero menos apta para tareas complejas como lo es `gpt-4.1-nano`. Este modelo arrojó buenos resultados y también era de los más económicos disponibles.

8.3.2. Arquitectura interna

Para escuchar solicitudes, `docs-gpt` esta suscrito a un canal llamado `docs_gpt` a través del sistema pub/sub de Redis. A través del mismo recibe las consultas para enviarlas luego a OpenAI. Los 2 mensajes principales que se procesan son los siguientes:

- **Edición parcial** (`PARTIALGEN_REQ`): Va a recibir como contexto el texto seleccionado por el usuario, donde el LLM deberá generar un nuevo texto modificado con el prompt del usuario y el contexto.

- **Edición completa** (`FULLGEN_REQ`): El usuario proporciona nada más que el prompt, y el microservicio `docs-gpt` recupera como contexto el documento completo directamente desde Redis para que el LLM genere un nuevo contenido total.

8.3.3. Solución multithreading adoptada

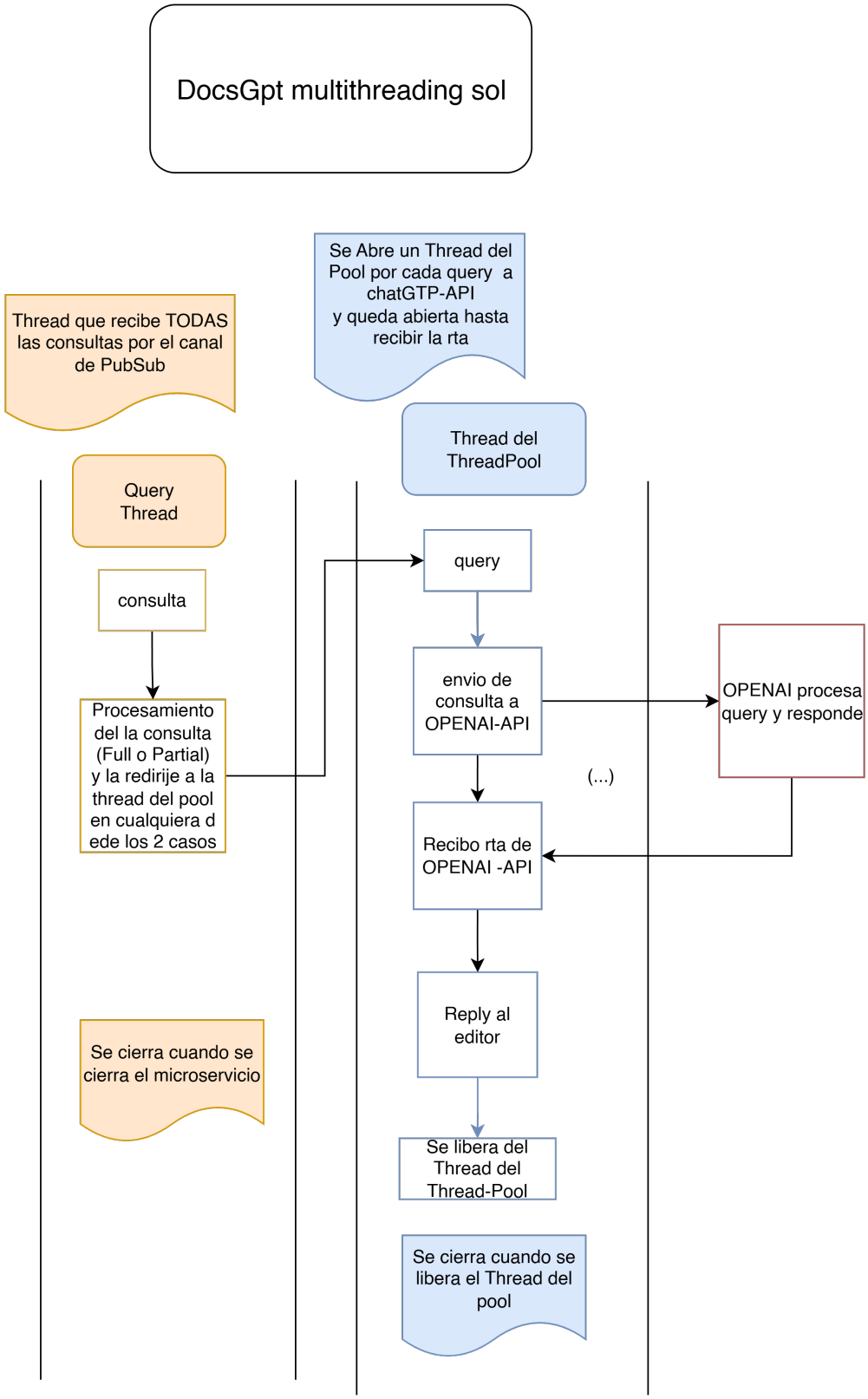

Figura 15: Flujo de procesamiento de consultas de generación en el microservicio

Para procesar estas peticiones, emplea un `thread pool` de 8 hilos que permite manejar múltiples solicitudes en paralelo. La lógica de procesamiento está desacoplada mediante un canal interno de mensajes, que recibe eventos del tipo `DocsGptAction` y los encola para su ejecución a medida que los workers de la `thread pool` se vayan desocupando. Una vez obtenida la respuesta del proveedor, el microservicio publica el contenido generado en el canal correspondiente al documento en cuestión, con su ID de cliente correspondiente de modo que pueda ser consumido por el cliente y se libera la thread del pool para poder procesar una nueva consulta a la API de OpenAI.

8.3.4. Uso de la API de OpenAI

El microservicio realiza una llamada HTTP POST al endpoint de OpenAI usando el modelo `gpt-4.1-nano`. El cuerpo del mensaje enviado contiene tres mensajes estructurados como contexto para el modelo se define de la siguiente forma:

```
let mut res = ureq::post(OPENAI_API_ENDPOINT)
    .header("Authorization", auth_header_val)
    .send_json(serde_json::json!({
        "model": OPENAI_MODEL,
        "input": [
            { "role": "system", "content": include_str!("../generate
                .prompt") },
            { "role": "user", "content": user_prompt },
            { "role": "user", "content": doc_content }
        ],
    }))
    .unwrap();
```

1. **System prompt**: Define el rol del asistente como un corrector y generador de texto integrado en un editor. El contenido de dicho prompt puede encontrarse en el archivo `docs-syncer/generate.prompt`.
2. **Instrucción del usuario**: El `prompt` proporcionado directamente por el cliente.
3. **Contenido del documento**: Enviado por el cliente (para generación parcial) o recuperado de Redis (para generación completa).

La respuesta del modelo es limpiada de caracteres especiales y formateada adecuadamente antes de ser enviada de vuelta al cliente.

8.4. Editor de texto colaborativo

La interfaz de usuario de la aplicación cliente, desarrollada utilizando el framework `eframe` con la librería `egui`, está diseñada para la gestión y edición colaborativa de documentos de tipo texto y planillas de cálculo.
Esta pantalla es la primera interacción del usuario con la aplicación y ofrece funcionalidades para descubrir y crear documentos.

- **Visualización de Documentos Guardados:**
  - Al inicializarse, el `Menu` consulta al almacén de datos para obtener la lista de todos los documentos existentes, incluyendo sus IDs, nombres base y marcas de tiempo de última edición.
  - Los documentos se muestran en una lista ordenada por la fecha de última edición (los más recientes primero), permitiendo al usuario seleccionar un documento existente para editarlo.
- **Creación de Nuevos Documentos:**

- Permite al usuario introducir un nombre para un nuevo documento. La aplicación valida la extensión del archivo para determinar el tipo de documento (ej., `.txt` para texto, `.xsl` para hojas de cálculo).
- Al crear un documento, el `Menu` **publica un mensaje al canal** `"docs_syncer"` **de Redis**. Este mensaje contiene los metadatos del nuevo documento (ID y nombre). Esta acción notifica al microservicio `DocsSyncer`, el cual es el responsable de inicializar y persistir el nuevo documento en el almacenamiento.

8.4.1. Pantalla de edición

La pantalla `Editor` es donde ocurre la edición colaborativa de documentos.

- **Inicialización y Suscripción:**
  - Al abrir un documento, el `Editor` genera un `client_id` independientemente a que documento se conecte .
  - Inmediatamente, **se suscribe a un canal Pub/Sub de Redis específico para el documento** (`doc_id`). Esta suscripción se realiza en un hilo separado para escuchar de forma asíncrona las actualizaciones y mensajes de coordinación relacionados con el documento.
  - Envía un mensaje `FETCH_REQ` al canal del documento para solicitar su contenido inicial al `DocsSyncer`.
- **Gestión de Contenido y Colaboración:**
  - El `Editor` maneja el contenido del documento, que puede ser texto plano o una hoja de cálculo.
  - Procesa mensajes recibidos del canal del documento: `FETCH_ACK`, *CLIENTS*, `PATCH`, `FULLGEN_ACK`, `PARTIALGEN_ACK`
- **Persistencia de Cambios:**
  - Cuando el usuario hace clic en "Guardar", el `Editor` construye un mensaje `PATCH` que contiene el `client_id`, el tipo de documento y el contenido actual.
  - **Publica este mensaje al canal del documento en Redis**. Esta acción notifica al `docs_syncer` que hay cambios pendientes para persistir en la base de datos.
- **Manejo de Desconexiones:** Al salir del editor (botón "Volver"), se envía un mensaje `LEAVE` al canal del documento para notificar al `docs_syncer` que el cliente se ha desconectado.

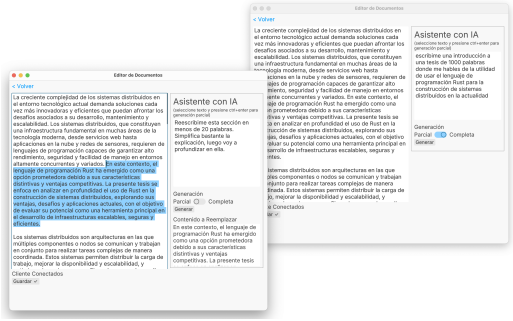

Figura 16: Demostración del editor de documentos en uso.

8.4.2. Generación de contenido

Se agregó una sección en la vista de edición de documentos de texto para enviar prompts a través del microservicio.

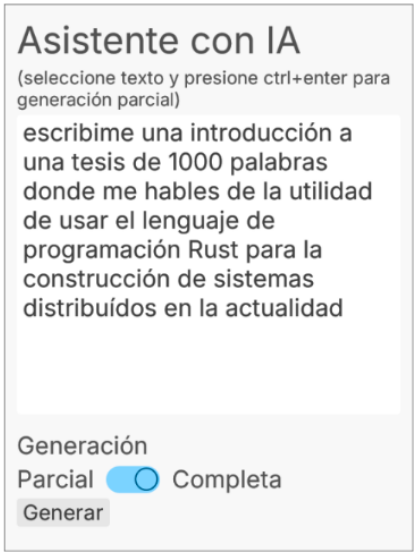

Figura 17: Seccion donde se escribe la prompt que se le hara al LLM a traves del microservicio

La entidad **Editor** es responsable de mantener un control único sobre el estado del documento que se encuentra en edición. En este caso, se incorporaron los campos `prompt`, `selected_range` y `full_gen_mode`, con el objetivo de añadir soporte para la generación de contenido. Asimismo, se agregaron variables auxiliares destinadas a gestionar el estado de la generación, permitiendo determinar su validez y si se encuentra pendiente.

Se puede seleccionar un rango de texto en el buffer activo o colocar el cursor en cualquier parte del documento para insertar o reemplazar contenido. En el caso de seleccionar un rango, el contenido seleccionado es enviado como contexto a la IA. Para hacer ello se debe presionar `ctrl+enter`.

9. Rendimiento y costos de funcionamiento

Al finalizar la construcción del microservicio de generación de contenido, pusimos a prueba el rendimiento del editor y ambos microservicios, usandolos en un caso de uso real. En este ejemplo, mantuvimos una sesión de edición de aproximadamente 10 minutos en la que editamos secciones este mismo informe mientras la redactábamos.

Utilizamos comandos de shell del sistema operativo para ir guardando los datos de consumo de CPU de los diferentes procesos para luego graficarlos.

Las pruebas fueron hechas en un procesador Apple M1 Pro de 8 núcleos, utilizando la estructura de cluster propuesta por la consigna y tres editores clientes, que alternaban entre diferentes documentos a la vez.

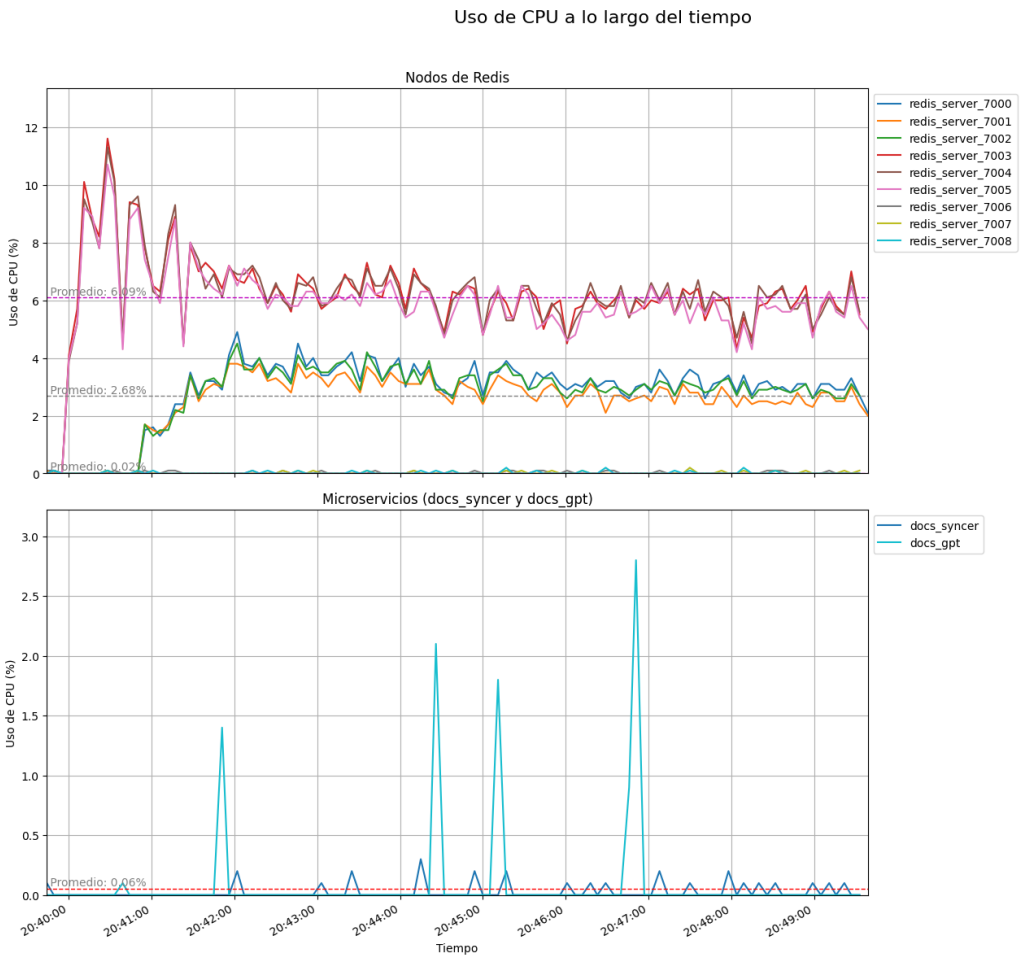

Figura 18: Rendimiento de CPU en caso de uso simulado.

Se puede observar que se distinguen tres grupos de nodos, con usos diferentes a lo largo de la demostración. El grupo que se encuentra más por encima del resto es el constituído por los nodos que pertenecen al tercer shard (nodo 7006) del cluster propuesto. Justamente en donde están almacenados los ids de los documentos y los timestamps de las modificaciones a los mismos. Este es el conjunto de nodos con mayor actividad debido a que todas las operaciones de persistencia pasan por ellos. En la etapa inicial, cuando los clientes se conectan al servidor, son quienes más carga tienen al suplir la información inicial a cada cliente, llegando a alcanzar aproximadamente un 12 % de consumo de CPU, para luego estabilizarse alrededor del 6 %.

El segundo grupo corresponde a los nodos a los que se conectaron los editores de texto directamente. La actividad de estos nodos se estabilizó alreadedor del 2.7 % del consumo de CPU. Finalmente, el último grupo, fueron aquellos nodos que no se encargaron de nada más que de hacer broadcast de los mensajes pub/sub y de la funcionalidad de cluster. Estos nodos estuvieron prácticamente en idle a lo largo del ejemplo, y su consumo fue mínimo, ubicándose en alrededor de 0.02 % del uso de CPU.

En cuanto al consumo de recursos de los microservicios, el mismo fue todavía inferior a de los servidores, promediando un 0.06 % de utilización del CPU. Los únicos momentos en los que ascendió este consumo fue cuando se solicitaron grandes generaciones de contenido de varios documentos en simultáneo. De igual forma, el consumo nunca sobrepasó el 3 %.

Por otra parte, revisamos las métricas del consumo de créditos de OpenAI durante la

demostración dicha y el período de construcción de esta funcionalidad.

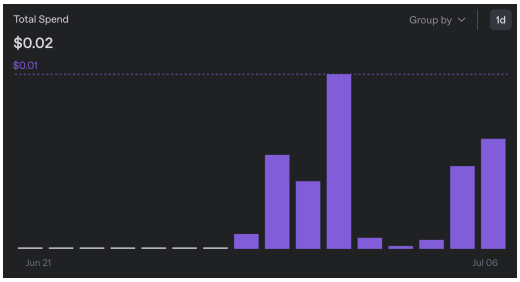

Figura 19: Consumo de créditos de OpenAI API durante la primera semana final de elaboración del proyecto.

A lo largo de la semana de elaboración de esta feature, y después de todo el testeo manual que hicimos, probando generaciones de ambos tipos, con contenido de diversas longitudes, terminamos gastando solamente el equivalente a USD$0.02, habiendo consumido 51.861 tokens en total. Esto ratifica nuestra elección de modelo, que al ser un modelo relativamente pequeño, tiene un consumo mucho menor al resto.

Debido al bajo consumo de memoria que tienen los procesos (alreadedor de 5 MB durante la ejecución de la demo, y alrededor de 4 MB en idle). Estimamos que si quisieramos utilizar un servicio de cloud para ejecutar el cluster, lo que más nos convendría seria utilizar un VPS. Para el caso de uso de este proyecto en particular, los droplets de Digital Ocean, específcamente el de menor costo, son más que suficientes para mantener hostear el proyecto.

En cuanto a consumo del proveedor de IA, el mismo va a depender significativamente del tipo de documentos que se edite y de la cantidad de solicitudes que se le haga al proveedor. Actualmente los modelos de LLM son cada vez más económicos, por lo que la edición de documentos extensos y la constante solicitud de generación no debería ser muy costosa. Habría que evaluar bien cuál sería el perfil de usuarios y documentos que se utilizarían con el servicio para establecer un presupuesto.

10. Aprendizajes obtenidos

El desarrollo de este sistema de edición colaborativa nos aporto mucho conocimiento acerca de ingeniería de software, sistemas distribuidos y concurrencia. A continuación, se detalla un poco más:

- Diseño e implementación de protocolos de comunicación siguiendo estándares ya definidos. La implementación de protocolo RESP y de las respuestas estándar de los comandos de Redis seleccionados nos permitió aprender a seguir especificaciones. Al hacer nuestra implementación spec-compliant, pudimos utilizar clientes oficiales de Redis para probar nuestra propia implementación.

- Modelo de actores concurrente para evitar compartir estado entre diferentes threads. Este patrón se utilizó de manera extensiva en el servidor e incluso en las aplicaciones cliente. Nos permitió evitar el uso de mutex en varias situaciones en donde, a primera vista, el modelo de concurrencia con mutexes nos perecía mas intuitivo. La transmisión de la propiedad de los datos a través de los canales elimina la necesidad de bloqueos explícitos, reduciendo la complejidad del código concurrente y mitigando riesgos de *deadlocks* y mejorando el rendimiento de la aplicación.

- Utilización de pub/sub como servicio de mensajería en sistemas distribuídos. Como mencionamos anteriormente, este proyecto fue la primera vez que todos los

integrantes del grupo construímos sistemas distribuídos. Este patrón fue el pilar de la comunicación en tiempo real entre la UI y los microservicios, permitiendo la propagación instantánea de eventos (creación de documentos, actualizaciones de contenido, cambios en la lista de colaboradores) a través de los canales de Redis. Aprendimos que este mecanismo de propagación de mensajes nos permite mantener una comunicación stateless, e incluso nos terminó resultando intuitivo.

- Se aprendió a interactuar con una base de datos distribuida configurada en modo clúster. La implementación incluyó la lógica necesaria para manejar las redirecciones inherentes a un Redis Cluster. Esto aseguró que el cliente editor y los microservicios pudieran conectarse y operar correctamente, incluso cuando las claves residían en diferentes nodos del clúster, lo cual es vital para la robustez en entornos de producción distribuidos.

- Conocimiento sobre los algoritmos de consenso. Este fue un tema que se nos dificultó mucho, ya que inicialmente nuestra idea fue implementar un algoritmo de consenso basado enteramente en el que utiliza la implementación oficial de Redis. Sin embargo, tras consultar con nuestro corrector, él mismo nos comentó que los algoritmos de consenso son temas que naturalmente son muy complejos, tanto que incluso hay una materia dedicada a ellos. Finalmente terminamos implementando nuestra propia versión simplificada de un algoritmo de consenso, lo que también nos enseñó mucho de esta familia de algoritmos, como el concepto de épocas y manejo de casos de empate.

- Aprendizaje Docker: Aprendimos a utilizar Docker para construir imágenes de Rust que generan binarios ejecutables dentro de un contenedor. También exploramos el uso de Docker Compose para conectar múltiples contenedores entre sí. Ademas hicimos uso de layers para reducir el tamaño final de la imagen, esto nos permitió incluir únicamente las dependencias necesarias para la ejecución del programa, excluyendo aquellas dependencias del entorno de desarrollo de compilación.